

第21回 勇者と敵を分ける:継承

今回のゴール

- `extends` を使って、`Character` を受けついだ `Hero` と `Enemy` を作れる。
- 子クラスで `attack` を **オーバーライド** し、勇者と敵で違う攻撃にできる。
- `super(...)` と `super.attack(...)` の意味を説明できる。

前回とのつながり

第20回までは、勇者もゴブリンも同じ `Character` クラスだったので、ゴブリンまで会心の一撃を出していた。また、薬草の数 `herbCount` は勇者だけの情報なのに `Main` に置かれていた。今回は、共通部分を `Character` に残したまま、勇者と敵それぞれの違いを別のクラスに書く。

実習時のフォルダ構成

```
ワークスペース
├─ java21_sample
│   └─ src
│       ├── Character.java
│       ├── Hero.java
│       ├── Enemy.java
│       └─ Main.java
└─ java21_exercise
    └─ src
        ├── Character.java
        ├── Hero.java
        ├── Enemy.java
        ├── Main.java
        └─ Dragon.java ← 演習2-2で自分で作る
```

- 配布ファイル: `プログラム/第21回/sample/`、`プログラム/第21回/exercise/`
- 解答例: `プログラム/第21回/answer/`

講義内容

1. 継承とは

```
public class Hero extends Character {
    ...
}
```

- `extends Character` と書くと、`Hero` は `Character` のフィールドとメソッドを すべて受けつぐ。`Hero` の中に書いていない `showStatus()` や `heal()` も、`hero.showStatus()` のように使える。
- 受けつぐ元を 親クラス(スーパークラス)、受けつぐ側を 子クラス(サブクラス) という。
- 子クラスには、その子クラスだけのフィールドやメソッドを追加できる(勇者の `herbCount`、敵の `exp`)。

クラス	役割	持っているもの
<code>Character</code>	全員に共通	名前、HP、攻撃力、ふつうの攻撃、回復、ダメージ
<code>Hero</code>	勇者だけ	薬草、会心の一撃
<code>Enemy</code>	敵だけ	経験値

2. 子クラスのコンストラクタと super(...)

プログラム/第21回/sample/Hero.java

```
// 勇者: Character の機能をすべて受けつぎ、薬草と会心の一撃を持つ
public class Hero extends Character {
    private int herbCount;

    public Hero(String name, int hp, int power, int herbCount) {
        super(name, hp, power);    // 親クラス (Character) のコンストラクタを呼ぶ
        this.herbCount = herbCount;
    }

    public int getHerbCount() {
        return herbCount;
    }

    // 薬草を使う
    public void useHerb() {
        if (herbCount > 0) {
            heal(15);
            herbCount--;
            System.out.println(getName() + "は薬草を使った！ HPが" + getHp() + "/" + getMaxHp() + "になった");
        } else {
            System.out.println("薬草がない！");
        }
    }

    // 勇者の攻撃: 3 ターンごとに会心の一撃 (1.5 倍)
    @Override
    public void attack(Character target, int turn) {
        if (turn % 3 == 0) {
            int damage = (int) (getPower() * 1.5);
            System.out.println("会心の一撃！");
            System.out.println(getName() + "の攻撃！ " + target.getName() + "に" + damage + "のダメージ");
            target.damage(damage);
        } else {
            super.attack(target, turn);    // 親クラスのふつうの攻撃
        }
    }
}
```

- `super(name, hp, power);` は、親クラスのコンストラクタを呼び出す。名前・HP・攻撃力の準備は `Character` に任せ、`Hero` は薬草の数だけを準備する。コンストラクタの 1 行目 に書く。
- `name` や `hp` は `Character` で `private` なので、`Hero` の中からも直接は使えない。`getName()`、`getPower()` などの getter を使う。
- `heal(15)` のように、親クラスの `public` メソッドは、子クラスの中でそのまま呼べる。

3. オーバーライド

親クラスと 同名・同じ引数 のメソッドを子クラスに書くと、子クラスのインスタンスでは子クラスのメソッドが使われる。これを **オーバーライド** という。

呼び出し	実行されるメソッド
<code>hero.attack(enemy, turn)</code>	<code>Hero</code> の <code>attack</code> (会心の一撃あり)
<code>enemy.attack(hero, turn)</code>	<code>Enemy</code> には <code>attack</code> がないので、 <code>Character</code> の <code>attack</code>

- `@Override` は「親クラスのメソッドを上書きするつもりです」という印。名前や引数を間違えたとき、コンパイラがエラーで教えてくれる。
- `super.attack(target, turn)` は、「親クラスの `attack`」を呼ぶ。会心の一撃でないターンは、ふつうの攻撃をそのまま使っている。
- `Character` の `attack` は `turn` を使っていないが、子クラスで上書きできるように、同じ形の引数にそろえている。

プログラム/第21回/sample/Enemy.java

```
// 敵：Character の機能をすべて受けつぎ、たおすともらえる経験値を持つ
public class Enemy extends Character {
    private final int exp;

    public Enemy(String name, int hp, int power, int exp) {
        super(name, hp, power);
        this.exp = exp;
    }

    public int getExp() {
        return exp;
    }
}
```

プログラム/第21回/sample/Main.java (変わったところ)

```
Hero hero = new Hero("勇者", 30, 8, 2);
Enemy enemy = new Enemy("ゴブリン", 40, 7, 12);
```

```
case 2:
    hero.useHerb();
    break;
```

```
System.out.println(enemy.getName() + "をたおした！ 経験値" + enemy.getExp() + "を手に入れた");
```

- 薬草の処理が `Hero` に移ったので、`Main` の `case 2` は 1 行になった。
- `Character.java` は、第20回の解答例とほぼ同じ。`attack` から会心の一撃を取り除き、ふつうの攻撃だけにしている。

4. 実行する前に予想する

1、1、1、2、1、1 の順に入力する。

- 3 ターン目、ゴブリンは会心の一撃を出すか。
- 勝ったときの表示はどうなるか。

5. 実行結果(3 ターン目と最後)

```
--- ターン3 ---
勇者 HP:16/30
ゴブリン HP:24/40
1: 攻撃
2: 薬草を使う
3: にげる
4: ためる
コマンドを選ぶ > 1
会心の一撃！
勇者の攻撃！ ゴブリンに12のダメージ
ゴブリンの攻撃！ 勇者に7のダメージ
```

```
コマンドを選ぶ > 1
会心の一撃！
勇者の攻撃！ ゴブリンに12のダメージ

ゴブリンをたおした！ 経験値12を手に入れた
```

- 会心の一撃を出すのは勇者だけになった。

6. エラーを体験する

1. `Hero.java` の `@Override` の下のメソッド名を `atack` (t が 1 つ) に変えて保存する。
2. 「親クラスのメソッドをオーバーライドしていない」というエラーになることを確認する。`@Override` を付けていたおかげで、書き間違いに気付ける。
3. 元に戻す。次に `super(name, hp, power);` の行を消して保存し、エラーを確認する(`Character` には引数なしのコンストラクタがないため)。確認したら元に戻す。

演習1 サンプルを実行する

`java21_sample` でサンプルを実行し、予想と結果を比べる。

演習2 敵の攻撃を作る

`java21_exercise` を作り、`プログラム/第21回/exercise/` の 4 ファイルを入れる。

演習2-1 敵の強攻撃

- `Enemy` で `attack` をオーバーライドし、4 ターンごとに「強攻撃」(攻撃力の 2 倍)にする。表示は `ゴブリンの強攻撃！ 勇者に14のダメージ` の形にする。
- それ以外のターンは、`super.attack(target, turn)` でふつうの攻撃をする。

ヒント: `Hero` の `attack` と同じ形で書ける。攻撃力は `getPower()` で取り出す。

演習2-2 ドラゴンを作る

- `src` を右クリックして、新しいクラス `Dragon` を作る。`Dragon` は `Enemy` を受けつぐ。
- コンストラクタは `Enemy` と同じ 4 つの引数を受け取り、`super(...)` に渡す。
- `attack` をオーバーライドし、奇数ターンは `ドラゴンは炎をはいた！ 勇者に11のダメージ` (攻撃力 + 5)、偶数ターンは `super.attack(target, turn)` にする。
- `Main` で、敵をドラゴン(HP 60、攻撃力 6、経験値 50)に変える。

ヒント:

- 奇数は `turn % 2 == 1`。
- `Main` では `Dragon enemy = new Dragon("ドラゴン", 60, 6, 50);` と書く。

完成条件(1 を続けて入力したとき):

```
コマンドを選ぶ > 1
勇者の攻撃！ ドラゴンに8のダメージ
ドラゴンは炎をはいた！ 勇者に11のダメージ

--- ターン2 ---
勇者 HP:19/30
ドラゴン HP:52/60
1: 攻撃
2: 薬草を使う
3: にげる
4: ためる
コマンドを選ぶ > 1
勇者の攻撃！ ドラゴンに8のダメージ
ドラゴンの攻撃！ 勇者に6のダメージ
```

4 ターン目には、偶数ターンなので `Enemy` の `attack` が呼ばれ、`ドラゴンの強攻撃！` になる。`Dragon` の `super.attack` が、`Enemy` の `attack` を呼んでいることを確かめる。

発展(任意)

- ドラゴンはとても強い。薬草と「ためる」を使って勝てるか試す。勝てなければ、勇者の薬草の数を変える。
- `Slime` (スライム)クラスを作り、たまに「なかまを呼ぶふりをする(何もしない)」攻撃にする。

確認問題

問1 `new B().hello();` の出力は何か。

```
class A {
    void hello() {
        System.out.println("A");
    }
}

class B extends A {
    @Override
    void hello() {
        super.hello();
        System.out.println("B");
    }
}
```

問2 `new Child();` の出力は何か。

```
class Parent {
    Parent() {
        System.out.println("親");
    }
}

class Child extends Parent {
    Child() {
        System.out.println("子");
    }
}
```

問3 次の `Enemy` はコンパイルできるか(`Character` はサンプルと同じ)。

```
public class Enemy extends Character {
    public Enemy(String name, int hp, int power) {
        super(name, hp, power);
    }

    @Override
    public void attack(Character target, int turn) {
        System.out.println("攻撃");
    }
}
```

■ 答え

- 問1: `A` と `B` (`super.hello()` で親の `hello` を実行してから、`B` を表示する)
- 問2: `親` と `子` (子クラスのコンストラクタは、最初に親クラスのコンストラクタを呼ぶ。`super()` を書かなくても、自動で引数なしの `super()` が呼ばれる)
- 問3: できない (`attack` は親クラスにないメソッドなので、`@Override` がエラーになる)

振り返り

- 継承で、どこを `Character` に残し、どこを `Hero` と `Enemy` に分けたか説明できるか。
- オーバーライドしたとき、どちらの `attack` が実行されるか予想できたか。
- `super(...)` と `super.attack(...)` の違いを説明できるか。

次回へのつながり

今は勇者 1 人で戦っている。次回は魔法使いを仲間に加えてパーティーにする。勇者と魔法使いは違うクラスなのに、「全員、攻撃！」とまとめて命令できるようにする。